



STRICTLY CONFIDENTIAL

PROJECT DOCUMENT · V1.0

MatchHire *AI-Powered* Job Portal Platform.

A curated, intelligent career marketplace where senior talent meets the companies smart enough to hire them — reimagining recruitment through automated matching, transparent signals, and a calmer experience for both sides of the market.

PREPARED BY

MatchHire Inc.

DOCUMENT DATE

17 May 2026

DOCUMENT TYPE

Business & Technical

Table of *contents.*

DOCUMENT MAP · 20 SECTIONS

01	Executive Summary	BUSINESS
02	Project Vision	BUSINESS
03	Business Objectives	BUSINESS
04	Platform Overview	PRODUCT
05	User Roles & Permissions	PRODUCT
06	Complete User Flow	PRODUCT
07	Functional Modules	PRODUCT
08	AI Features	TECHNICAL
09	Technical Architecture	TECHNICAL
10	Recommended Technology Stack	TECHNICAL
11	Database Planning	TECHNICAL
12	Security Considerations	TECHNICAL
13	Scalability Planning	TECHNICAL
14	Development Roadmap	OPERATIONAL
15	Estimated Timeline	OPERATIONAL

16	Estimated Team Structure	OPERATIONAL
17	Future Enhancements	STRATEGIC
18	Conclusion	STRATEGIC
19	Appendix · Suggested APIs	APPENDIX
20	Appendix · Deployment Notes	APPENDIX

Executive *summary.*

MatchHire is an AI-powered job marketplace built for the next generation of professional hiring. The platform connects vetted senior talent with employers who value quality over volume, replacing the noise of conventional job boards with a calmer, signal-rich experience for both sides of the market.

Overview

The platform brings together three distinct surfaces — a Candidate Hub for job seekers, a Company Hub for employers, and an Admin Console for platform operators — each tailored to the workflows of its primary user. Underpinning these surfaces is a proprietary matching engine that scores candidates against opportunities across compensation fit, skills alignment, location, career trajectory, and cultural signals.

Purpose

MatchHire exists to remove the friction that has accumulated in modern hiring: vague job descriptions, irrelevant matches, opaque salary bands, slow response times, and the volume problem that buries qualified candidates beneath unqualified ones. The product expresses a clear philosophy — fewer, better matches, with the data to back every recommendation.

Business goals

- Establish MatchHire as the destination of choice for senior, hard-to-fill technical and creative roles.
- Reduce average time-to-hire from the industry baseline of 42 days to under 21 days for partner employers.
- Achieve a 90%+ match-relevance score across the top three recommendations served to any active candidate.
- Build a recurring revenue base through tiered employer subscriptions and premium candidate services.

Market opportunity

The global online recruitment market is projected to surpass **USD 43 billion by 2027**, with the highest growth in specialised, AI-augmented hiring platforms targeting senior and technical roles. MatchHire enters this market with a differentiated thesis: curation over volume, signal over advertising, and intelligence over keyword search. The opportunity is not to compete with mass job

boards, but to displace the recruiter-led hiring funnel for a clearly defined segment of high-trust roles.

POSITIONING STATEMENT

A premium, intelligent career marketplace — where every match is earned, every recommendation is explained, and every interaction respects the time of both parties.

Project *vision.*

To make hiring feel less like advertising and more like introduction — where the right person and the right company find each other quickly, transparently, and with mutual respect for the decision in front of them.

Long-term vision

Over the next five years, MatchHire intends to evolve from a curated job marketplace into a complete **career operating system** — owning the candidate journey from discovery, through application, through interview preparation, to negotiation and onboarding. On the employer side, the platform will expand from a sourcing channel into a full applicant-tracking and hiring-intelligence suite, with predictive insights that help hiring teams forecast offer acceptance, compensation positioning, and retention risk before extending an offer.

AI hiring transformation

Conventional hiring tools treat AI as a filter — a way to discard résumés faster. MatchHire treats AI as a **recommender** — a way to surface the right people and the right roles to the front of each other's queue. Every algorithmic decision carries a human-readable explanation, and every recommendation invites a counter-factual ("show me roles that are more remote-friendly", "show me candidates with stronger backend experience"). The system learns from both signals and corrections, improving over time without becoming a black box.

Recruitment automation goals

- **Automate the obvious.** Skill extraction, salary normalisation, application acknowledgement, interview scheduling, and reference outreach should run without human attention.
- **Augment the judgement-heavy.** Shortlisting, evaluation, and offer crafting should be assisted — never decided — by the platform.
- **Eliminate ghosting.** Every candidate receives a definitive response within a published service-level agreement; every employer receives a structured signal of candidate intent.
- **Compress the cycle.** The end-to-end median hiring cycle should be measurable, comparable, and shrinking quarter on quarter.

Business *objectives.*

Five concrete objectives anchor the first eighteen months of operation. Each is measurable, owns a primary metric, and ladders up to the platform's overarching commercial thesis.

①

Faster hiring

Reduce median time-to-offer by 50% versus the industry baseline through automated screening, instant matching, and integrated scheduling.

②

Better candidate matching

Deliver match scores that correlate with downstream interview-conversion rates above 70% — a measurable, predictive signal rather than a marketing number.

③

Reduced recruitment cost

Cut employer cost-per-hire by 40% relative to agency-led sourcing by eliminating contingent recruiter fees and shortening the funnel.

④

Improved hiring workflow

Replace fragmented tooling (job boards, ATS, scheduler, reference checks) with a single, integrated workflow that reduces context switching.

⑤

Employer and candidate engagement

Sustain a weekly active rate above 60% on both sides of the marketplace through intelligent notifications, transparent application status, and a steady stream of relevant new opportunities or candidates.

Objective metrics

OBJECTIVE	PRIMARY METRIC	TARGET (YEAR 1)
Faster hiring	Median time-to-offer	≤ 21 days
Better matching	Top-3 match → interview rate	≥ 70%

OBJECTIVE	PRIMARY METRIC	TARGET (YEAR 1)
Lower cost	Employer cost-per-hire	40% reduction vs. baseline
Workflow	Tools consolidated per hire	From 4–6 → 1
Engagement	Weekly active users	≥ 60% of activated accounts

Platform *overview.*

MatchHire is composed of ten interlocking modules, each designed to serve a distinct user need while contributing data and signal to the others. Together they form a coherent product surface that is simple at the edges and intelligent at the core.



Candidate Portal

A dedicated workspace for job seekers — profile builder, job search, application tracker, interview calendar, and personalised recommendations.



Employer Portal

Company-side hub for posting roles, reviewing applicants, managing interview pipelines, and tracking funnel performance against benchmarks.



Admin Console

Platform-operator surface providing verification queues, content moderation, user management, and system health monitoring.



AI Matching System

Proprietary scoring engine that ranks candidate-role compatibility across compensation, skills, location, stage, and cultural fit.



Dashboard System

Role-aware analytics dashboards that surface the right metric for the right user — applications for candidates, funnel health for employers, platform vitals for admins.



Notifications

Multi-channel delivery (in-app, email, push) with intelligent volume control — five great matches a day rather than fifty mediocre ones.



Favorites & Preferences

Collections, deal-breakers, and weighted preferences let candidates teach the engine what good looks like and refine matches in real time.



Job Recommendations

Continuously refreshed feed of the highest-scoring open roles, with transparent explanations behind every suggestion.



Company Profiles

Verified employer pages with culture signals, team size, funding stage, and aggregate engagement metrics from past hires.



Resume Management

Structured profile data with optional file uploads, automatic parsing, and version history — keeping candidates' presentation polished without manual rework.

User roles & *permissions*.

Access control follows a strict role-based access control (RBAC) model. Every privileged action checks both the user's role and the relationship between the user and the target resource. Permissions are enforced server-side and never inferred from UI state.

ROLE	PRIMARY CAPABILITY	SCOPE
Candidate Self-service	Build profile · apply to roles · manage favorites · configure preferences · track applications · interview scheduling.	Own profile and applications only.
Employer Team-scoped	Post jobs · review applicants · shortlist candidates · schedule interviews · access hiring funnel analytics · manage company profile.	Company-owned content; teammates within the same company.
Admin Operational	Approve company verifications · review flagged content · moderate users · respond to reports · access platform-wide analytics.	Read/write across moderation surfaces; read across users and companies.
Super Admin Platform owner	All Admin powers plus: configure platform settings · manage admin team · access financial reporting · approve product-level releases.	Unrestricted within the platform. Limited to a small, audited team.

PRINCIPLE

Authority follows responsibility. The principle of least privilege is enforced by default: a Candidate cannot see another candidate's profile, an Employer cannot see candidates who have not applied to one of their roles, and an Admin can never read private candidate-employer communication without explicit consent.

Complete *user flow*.

The platform supports two distinct journeys — Candidate and Employer — that converge at the Application moment. Below is the end-to-end choreography for both sides, alongside a visual map of the workflow.

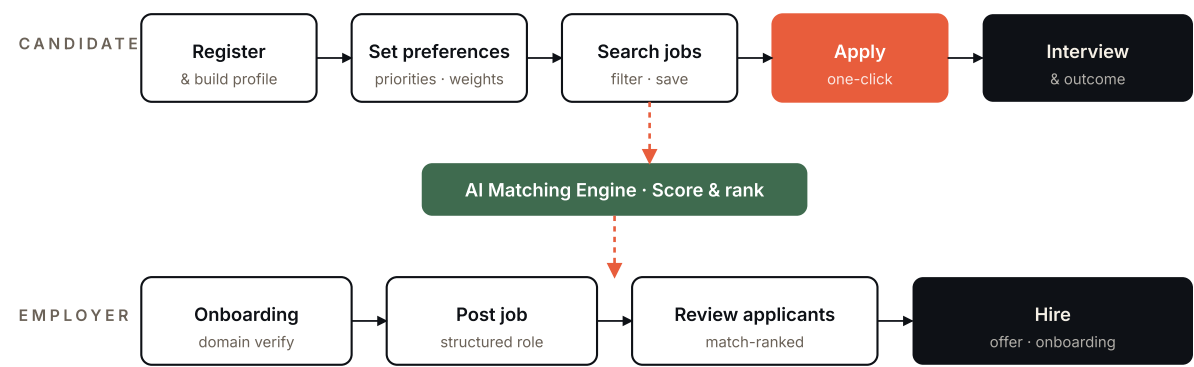
Candidate journey

1. **Registration** — Account creation via email or OAuth. Initial role selection (job seeker vs. employer).
2. **Profile completion** — Guided builder for personal information, work experience, skills, and target role. Completion percentage is shown live to drive the user to 100%.
3. **Preferences** — Candidate ranks their top priorities (compensation, growth, balance, etc.), sets salary expectations, work-mode and location preferences, and configures dealbreakers.
4. **Job search** — Browse the full open roles index or rely on personalised recommendations. Filter by experience level, salary band, work mode, and skill stack.
5. **Apply flow** — One-click apply using the structured profile. Optional cover note and answer to employer-defined screening questions.
6. **Interview & outcome** — Receive interview invitations directly in the calendar; track status as the application progresses through the employer's pipeline; receive a definitive outcome within the published SLA.

Employer journey

1. **Employer onboarding** — Submit company information (domain, industry, headcount), pass domain verification, and configure billing.
2. **Job posting** — Compose a structured role with required and preferred skills, compensation band, location, and screening questions.
3. **Candidate review** — Inbound applicants are ranked by match score. Employers review profiles, shortlist, request additional information, or reject with a templated reason.
4. **Hiring workflow** — Shortlisted candidates move through the configurable pipeline (Reviewed → Interview → Offer → Hired) with full funnel visibility and historical benchmarks.

FIGURE 6.1 – END-TO-END PLATFORM WORKFLOW



Functional *modules*.

Each module owns a clearly defined slice of the product surface, exposes a stable internal API to neighbouring modules, and can be developed, tested, and released independently. This separation is the foundation of the platform's long-term maintainability.

Core modules

MODULE	RESPONSIBILITIES
Authentication	Email + password, OAuth providers (Google), JWT issuance and rotation, password reset, multi-factor authentication for privileged roles.
Job Management	CRUD over job postings, status lifecycle (draft → active → paused → closed), structured fields (compensation, location, requirements), expiry handling, and audit trail.
Candidate Management	Profile data ownership, skill catalogue, work history, completion scoring, and visibility controls (public / recruiter-only / private).
Company Management	Company entity lifecycle from creation through verification, team management, billing entity linkage, and public profile rendering.
Search & Filters	Full-text search across jobs, companies, and candidates with structured facet filters (location, salary, experience, stack), backed by a dedicated search index.
Dashboard Analytics	Role-aware dashboards: candidate hub (applications, matches, profile health), company hub (funnel, applicants, posting performance), admin (platform vitals, revenue).
Preferences System	Ranked priorities, weighted factors, dealbreaker filters, notification thresholds, and per-user preference history.
Favorites System	Saved-jobs collections, per-collection labels, expiry reminders, and pattern insights derived from the saved set.

MODULE	RESPONSIBILITIES
Notifications	Multi-channel delivery (in-app, email, push), digest cadence control, per-event opt-in, and unread-state tracking.
Reporting	Scheduled and on-demand reports for employers (funnel performance, time-to-hire) and the platform team (revenue, engagement, content health).

AI *features.*

Artificial intelligence is not a feature of MatchHire — it is the platform's central proposition. The system applies machine learning at five distinct points in the user journey, each producing a transparent, auditable signal rather than an opaque verdict.



Smart candidate matching

For every active role, the engine produces a ranked list of candidates with a per-candidate match score broken down by the underlying contributing factors.



Resume screening

Uploaded resumes are parsed into structured fields, normalised against the platform's skill ontology, and surfaced to candidates for confirmation before they go live on a profile.



Skill analysis

Embeddings are computed for both skill descriptions and role requirements, enabling fuzzy matching that recognises "Postgres" and "PostgreSQL" as the same competency.



Recommendation engine

A continuous feed of relevant new roles for every candidate, scored against their preferences, weighted by recency, and explained in plain language.



Match scoring

Every recommendation carries a numeric match score (0–100) and a structured explanation across five dimensions: compensation fit, skill alignment, location compatibility, company stage, and career trajectory. Candidates can drill into the breakdown of any score; employers can do the same on the candidate side.

Scoring methodology

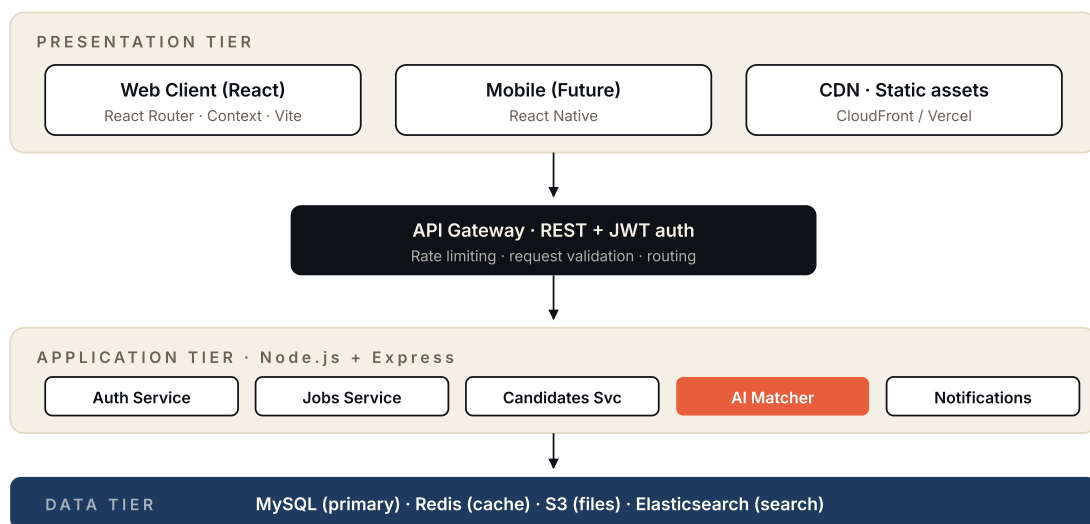
DIMENSION	WEIGHT (DEFAULT)	INPUTS
Skills match	30%	Required vs. possessed skills · embedding similarity · years of practice
Compensation fit	25%	Posted band vs. candidate expectation · currency normalisation
Career trajectory	20%	Seniority level · growth path implied by the role
Location compatibility	15%	Work-mode preference · timezone overlap · visa requirements
Company stage fit	10%	Stage preference · industry alignment · cultural signals

Weights are tuned per-candidate via the Preferences module. The defaults above represent the platform's average configuration.

Technical *architecture.*

MatchHire is architected as a modern, three-tier web application with a clear separation between presentation, application, and data layers. The system is built for clarity first, then for scale — every component can be horizontally replaced as load grows.

FIGURE 9.1 — SYSTEM ARCHITECTURE OVERVIEW



Frontend architecture

The web client is a single-page application built on React 18 and bundled with Vite. Routing is handled by React Router v6 in browser mode. Cross-cutting state lives in React Context (favorites, auth modal); page-local state stays inside its owning component. Styling uses a custom design system in plain CSS — no framework dependency — keeping the bundle small and the visual language tightly controlled.

Backend architecture

The application tier runs on Node.js with Express, organised as a modular monolith with service-oriented internal boundaries (Auth, Jobs, Candidates, Companies, Matching, Notifications). Each service owns its database tables and exposes a typed internal API. This layout converts cleanly to true microservices when individual modules cross the scaling threshold, without a disruptive rewrite.

Database structure

MySQL is the primary store, holding the authoritative state for all transactional data — users, jobs, applications, companies. Redis sits in front for hot reads (session lookups, leaderboard queries, rate-limit counters). Elasticsearch indexes job and candidate text for full-text search. S3 holds resumes, company logos, and other binary assets.

API architecture

The external API surface is a REST/JSON gateway, versioned at `/api/v1`. Every request is authenticated by JWT and authorised by the RBAC layer before reaching service logic. Response shapes are stable per major version; breaking changes ship behind a new version path with a published deprecation window.

Authentication flow

On successful sign-in the server issues a short-lived access token (15 minutes) and a longer-lived refresh token (14 days). The web client stores the refresh token in an HTTP-only, secure cookie and the access token in memory. The refresh exchange runs silently in the background; revocation is immediate on logout.

Deployment flow

Every merge to main triggers the CI pipeline (lint, type-check, test, build), produces immutable Docker images, and ships through staging into production. Infrastructure is provisioned on AWS with Nginx as the edge proxy, ECS or Kubernetes for compute, RDS for MySQL, ElastiCache for Redis, and CloudFront for the CDN.

Recommended *technology stack*.

The recommended stack favours well-understood, widely supported technologies. Each choice has a clear rationale: ecosystem strength, hiring availability, operational maturity, and a credible long-term roadmap.

LAYER	TECHNOLOGY	RATIONALE
Frontend	React 18 React Router v6 Vite	Industry-standard UI library with the largest component and tooling ecosystem. Vite delivers near-instant developer feedback.
Backend	Node.js LTS Express 4	Single language across the stack reduces context switching. Express remains the most battle-tested HTTP framework on Node.
Database	MySQL 8	Mature relational store with strong consistency, the right model for transactional hiring data. Read replicas scale horizontally.
Cache	Redis 7	Sub-millisecond reads for sessions, rate limits, and hot recommendation surfaces.
Search	Elasticsearch	Purpose-built for full-text search and faceted filtering across jobs, companies, and candidates.
Object storage	Amazon S3	Durable, cheap blob storage for resumes, logos, and exports.
Infrastructure	AWS Docker Nginx	Industry leader in cloud infrastructure with the broadest service catalogue. Docker and Nginx complete the stack at the deployment edge.
Authentication	JWT OAuth 2.0	JWT for stateless session tokens; OAuth for social sign-in (Google, GitHub).

LAYER	TECHNOLOGY	RATIONALE
AI / ML	Python (FastAPI) PyTorch · scikit-learn	The matching engine runs as a separate Python service to leverage the ML ecosystem; consumed by Node via internal REST.
Observability	Datadog · Sentry	Unified metrics, traces, and logs (Datadog) and frontend error tracking (Sentry).

Database *planning*.

The data model is anchored around eight primary entities. Relationships are expressed via foreign keys with strict referential integrity; audit fields (created_at, updated_at, deleted_at) appear on every table.

ENTITY	PURPOSE	KEY RELATIONSHIPS
Users	Authentication identity, role, contact details, account status.	1:1 with Candidate or Employer profile. 1:N with Sessions.
Jobs	Open role specification — title, description, compensation band, location, status.	N:1 with Companies. 1:N with Applications. M:N with Skills.
Applications	Candidate's expression of interest in a specific job, with pipeline status.	N:1 with Jobs. N:1 with Users (Candidate). 1:N with Interviews.
Companies	Employer entity — verified domain, industry, headcount, billing.	1:N with Jobs. 1:N with Users (employer team members).
Skills	Canonical skill ontology, normalised across job postings and candidate profiles.	M:N with Jobs (requirements). M:N with Users (possessed skills).
Preferences	Per-candidate ranked priorities, weighted factors, dealbreakers, salary expectations.	1:1 with Users.
Favorites	Saved-job records grouped into user-defined collections.	N:1 with Users. N:1 with Jobs.

ENTITY	PURPOSE	KEY RELATIONSHIPS
Notifications	Outbound message log across channels, with delivery and read state.	N:1 with Users.

Indexing strategy

- Composite index on jobs(status, created_at) for the home-page listing query.
- Composite index on applications(user_id, status) for the candidate's "my applications" view.
- Full-text index on jobs(title, description) mirrored into Elasticsearch.
- Unique index on users(email) and companies(domain).

Security *considerations.*

Security is treated as a product requirement, not an afterthought. The platform's threat model recognises that hiring data is personal, regulated, and commercially sensitive — its protection is a competitive necessity.



JWT authentication

Short-lived access tokens (15 min) signed with rotating keys; refresh tokens stored in HTTP-only secure cookies; immediate revocation on logout or compromise.



Role-based access control

Every privileged endpoint checks both role and resource ownership. UI hides ineligible actions; the server enforces them.



API request validation

Every inbound payload is validated against a strict schema; unknown fields are rejected; type coercion is explicit.



SQL injection prevention

Parameterised queries everywhere — no string-concatenated SQL. The data-access layer is the only path to the database.



XSS protection

React's default escape behaviour, strict Content Security Policy headers, and sanitisation of user-generated rich content.



Secure file uploads

MIME-type validation, virus scanning on ingest, signed URLs for retrieval, and per-tenant isolation in S3.

Compliance posture

- **GDPR.** Right to access, right to be forgotten, and data portability are first-class API features.
- **Data residency.** Region-pinned databases for jurisdictions with strict residency requirements.
- **Encryption.** TLS 1.3 in transit; AES-256 at rest for databases, backups, and object storage.
- **Audit trail.** All privileged actions are logged with actor, target, timestamp, and outcome.

Scalability *planning.*

The architecture is designed to absorb 100× load growth without a rewrite. Each tier scales independently, each bottleneck has a known remediation, and every architectural choice has been weighed against its scaling cost.

CAPABILITY	MECHANISM
Modular architecture	The application tier is organised as a modular monolith — distinct services internally, deployed as one binary. Internal boundaries are well-defined so they can split out when needed.
Microservice readiness	Auth, Jobs, Candidates, Notifications, and AI Matching are first-class internal services. Promotion to independent deployment is mechanical, not architectural.
Caching	Redis fronts session lookups, recommendation surfaces, rate-limit counters, and other hot reads. Cache invalidation is event-driven and explicit.
CDN readiness	All static assets ship to CloudFront with long TTLs and cache-busting hashes. The HTML shell itself is cacheable below a thin authentication edge.
Queue systems	SQS-backed queues for long-running work — resume parsing, bulk notifications, scheduled digests, and data exports — keeping request paths fast.
High-concurrency handling	Stateless application servers behind an autoscaling group. Database connection pooling via PgBouncer-equivalent for MySQL. Read replicas for high-volume reads.

SCALING PRINCIPLE

Scale the part that hurts, not the whole platform. The architecture lets the team identify the specific bottleneck and address it without a wholesale migration.

Development *roadmap*.

Delivery is structured into six sequential phases, each with a defined entry condition, a measurable exit criterion, and a working deliverable at the end. The phases overlap deliberately — backend foundations begin while the frontend is still in design polish, so dependent teams stay productive.

#	PHASE	SCOPE & DELIVERABLES
P1	UI / Frontend	Complete React SPA with all eleven routes, design system, and stub data. Production-grade visual treatment. Deliverable: deployable static site.
P2	Backend APIs	Express services for Jobs, Candidates, Companies, Applications. REST endpoints, validation, persistence. Deliverable: stable v1 API.
P3	Authentication	Email + OAuth sign-in, JWT issuance, role-based access control, session management. Deliverable: production-ready auth surface.
P4	AI Matching	Python service for skill embeddings, match scoring, recommendation feed. Deliverable: scored recommendations live on candidate dashboards.
P5	Admin Panel	Verification queue, content moderation tooling, user management, system health dashboards. Deliverable: operational platform.
P6	Testing & Deployment	End-to-end test suite, load testing, security review, production deployment, monitoring, runbooks. Deliverable: launched product.

Estimated *timeline.*

A 22-week delivery plan from project kickoff to production launch. Weeks overlap across phases to keep team utilisation efficient; milestones below mark the moments at which value becomes visible to stakeholders.

WEEKS	PHASE	KEY MILESTONES
W1 – W4	Phase 1 · UI / Frontend	Design system locked · all 11 routes scaffolded · clickable prototype shipped to stakeholders.
W3 – W8	Phase 2 · Backend APIs	Schema designed · Jobs/Candidates/Companies CRUD live · staging environment online.
W6 – W10	Phase 3 · Authentication	Email and OAuth sign-in working · RBAC enforced · sessions persisted.
W9 – W14	Phase 4 · AI Matching	Skill ontology seeded · match-scoring service deployed · recommendations live for beta users.
W12 – W17	Phase 5 · Admin Panel	Verification queue operational · moderation tools complete · admin onboarding documented.
W15 – W22	Phase 6 · Testing & Deployment	Full e2e suite passing · load testing complete · security audit closed · production launch.

RISK NOTE

The AI matching phase carries the highest delivery risk because of its dependence on a clean skill ontology. A two-week schedule buffer is included in phases 4 and 6 to absorb learning-curve overhead without compromising launch.

Estimated *team structure*.

An eleven-person core team can execute the 22-week plan with quality and predictable cadence. The structure is deliberately lean — senior engineers in primary roles, with quality-assurance, design, and operations distributed thoughtfully rather than scaled by headcount.

ROLE	HEADCOUNT	RESPONSIBILITIES
Frontend Developers	3	React SPA delivery, design-system implementation, accessibility, performance budgets.
Backend Developers	3	API services, data modelling, integration with AI service, observability instrumentation.
AI / ML Engineer	1	Embedding models, scoring engine, recommendation feedback loop.
UI/UX Designer	1	Information architecture, interaction design, visual system, prototyping.
QA Engineers	1	Test plans, automation, regression coverage, release gating.
DevOps	1	Infrastructure-as-code, CI/CD, monitoring, on-call rotations, security hardening.
Project Manager	1	Roadmap ownership, cross-team coordination, stakeholder communication, risk management.

Future *enhancements.*

The launch product establishes the foundation. The next twelve to eighteen months extend that foundation across new surfaces, new revenue models, and deeper intelligence. Each enhancement below has been sized as a self-contained programme that can be ranked, funded, and scheduled independently.



Video interviews

In-platform video calls with calendar integration, recording, transcription, and shareable highlights — eliminating a fragmented stack of external tools.



AI interview assistant

Real-time question suggestions for interviewers, post-interview summaries, and structured candidate-comparison briefs.



Real-time chat

Secure messaging between candidates and hiring teams with message-template support and full audit trail.



Mobile applications

Native iOS and Android apps built on React Native, sharing the design system and reusing the API surface from day one.



Subscription model

Tiered employer subscriptions (Starter · Growth · Enterprise) replacing pay-per-post, with premium analytics and seat-based pricing.



Premium hiring tools

Bulk outreach, sourcing intelligence, talent-pool nurture campaigns, and offer-acceptance forecasting — sold as Enterprise add-ons.

Conclusion.

MatchHire is positioned to redefine senior hiring by combining a calmer user experience with credible artificial intelligence, transparent signals, and an architecture built for measured growth.

The platform's product surface — Candidate Hub, Company Hub, Admin Console, AI Matching engine — is comprehensive enough to address the entire hiring lifecycle without forcing teams onto fragmented tooling. The technical foundation is deliberately mainstream: React, Node.js, MySQL, Redis, Elasticsearch, AWS. There is no novelty for novelty's sake; every layer is chosen because it is well-understood, well-supported, and trivial to staff.

The commercial proposition is equally clear. Specialised, AI-augmented hiring platforms are the fastest-growing segment of a USD 43 billion market, and MatchHire enters with a differentiated thesis — curation over volume, signal over noise, intelligence over keywords. Tiered subscriptions, premium tools, and adjacent revenue streams (assessments, interview tooling) compound the lifetime value of every employer relationship.

What follows the publication of this document is execution. The 22-week delivery plan and 11-person team structure described above are designed to put a production-grade platform in market by the end of the first half — at which point the work shifts from building to learning, and the data that the platform generates becomes its own competitive advantage.

CLOSING

The opportunity is real, the market is ready, the architecture is sound, and the team is sized to ship. The path from here is straightforward — build the product, measure the outcomes, refine continuously.

Suggested *APIs & integrations.*

A reference list of third-party services that accelerate delivery without compromising the long-term architecture. Each is replaceable on commodity infrastructure; the recommendation is to integrate them now and own them later, if and when scale justifies the engineering investment.

CAPABILITY	RECOMMENDED PROVIDER	NOTES
OAuth	Google Identity · GitHub OAuth	Standard social sign-in. Add Microsoft for enterprise.
Email delivery	SendGrid · Amazon SES	SendGrid for transactional and marketing. SES as a low-cost fallback.
SMS / push	Twilio · Firebase Cloud Messaging	Twilio for SMS at launch; FCM for mobile push when apps ship.
Payments	Stripe	Subscription billing, taxes, invoicing, and global payment methods.
Calendar & scheduling	Google Calendar · Microsoft Graph	Two-way sync for interview scheduling.
Video calls	Daily.co · Twilio Video	WebRTC infrastructure under the in-platform video product.
Maps & geocoding	Mapbox	Location normalisation, candidate-location matching, commute estimates.
Document parsing	Apryse · Adobe PDF Services	Resume extraction for candidates uploading PDF/DOCX files.
Embeddings	OpenAI · Cohere	Initial embedding source for skill and role text until in-house models are trained.
Background checks	Checkr	Pre-hire verification for premium employer tier.

CAPABILITY	RECOMMENDED PROVIDER	NOTES
Observability	Datadog · Sentry	APM, logs, traces (Datadog) and frontend exception capture (Sentry).

Deployment *notes.*

A pragmatic deployment baseline. Every assertion below has been chosen for operational simplicity at the early stage and a credible upgrade path as the platform matures.

Environments

- **Development** — Local Docker Compose stack covering MySQL, Redis, Elasticsearch, and the application services. One command to start (`make dev`), one to stop.
- **Staging** — Continuously deployed from `main`, mirroring production topology at reduced scale. Synthetic load and integration tests run on every deploy.
- **Production** — Multi-AZ deployment on AWS with autoscaling groups, RDS Multi-AZ MySQL, ElastiCache Redis, and CloudFront in front of the static surfaces.

CI / CD pipeline

1. Pull request opened → lint, type-check, unit tests, build.
2. Merge to `main` → integration tests, container image built and tagged, image pushed to ECR.
3. Automatic deploy to staging → e2e tests, smoke checks.
4. Promotion to production via gated approval → blue/green or canary rollout.
5. Post-deploy monitoring window with automated rollback trigger.

Backups & disaster recovery

- Daily MySQL snapshots with 30-day retention; weekly archival snapshots retained for one year.
- Point-in-time recovery enabled with a 7-day window.
- S3 versioning enabled on user-upload buckets.
- Quarterly disaster-recovery drill with full restore to an isolated environment.

SLA targets

METRIC	TARGET
API uptime (monthly)	99.9%
Mean response time (95th percentile)	< 300 ms

METRIC	TARGET
Mean time to acknowledge incident	< 5 minutes
Mean time to recover (sev-1)	< 60 minutes
Recovery point objective (RPO)	≤ 1 hour
Recovery time objective (RTO)	≤ 4 hours

END OF DOCUMENT · MATCHHIRE V1.0